16

Introduction

Linear algebra operations like the solution of linear systems, inversion and decomposition of matrices are arguably the most basic kind of numerical computations. Alongside even more elemental operations like matrix-vector and matrix-matrix multiplication, they are the building blocks of virtually all heavyweight computation on contemporary computers. Consequently, this area has been studied to great depth, which has led to extremely well-crafted algorithms. A thorough introduction can be found in a tome by Golub and Van Loan (1996). A more recent – and only slightly less voluminous – treatment is provided by Björck (2015). Just seeing these books, each well over 700 pages thick,[1] sitting on a bookshelf should convince anyone that it would be foolish to attempt a probabilistic analysis of all of numerical linear algebra in this chapter. As in the other chapters, we will focus on a few key aspects.

[1] A less hefty introduction at undergraduate level is offered by Ipsen's (2009) concise book.

One strong simplification we will make is to assume that computations can be performed with arbitrary precision. Doing so gives us a licence to ignore all questions of *numerical stability*. Seasoned numerical analysts may raise their eyebrows here – making algorithms stable to numerical errors is a core goal of research in numerical linear algebra. But in machine learning, machine precision is regularly by far the smallest source of uncertainty, dominated by issues arising from the lack of data and data-sub-sampling. By putting stability aside, we can concentrate on the question of epistemic uncertainty: what is the most efficient way to extract information about the solution of a linear system from a computer?

To simplify things further, we will focus on one particular problem – the solution of the symmetric, positive definite, real

https://doi.org/10.1017/9781316681411.022 Published online by Cambridge University Press

linear system (see Figure 16.1)

$$Ax = b, \quad \text{where } A \in \mathbb{R}^{N\times N} \text{ SPD., and } x, b \in \mathbb{R}^N. \tag{16.1}$$

Equation (16.1) can also be written as an optimisation problem: x is the unique vector that minimises the convex quadratic function

$$f(x) = \frac{1}{2}x^\intercal Ax - x^\intercal b, \tag{16.2}$$

and is thus known as *the least-squares problem*. $f(x)$ has gradient (see Figure 16.1)

$$r(x) := \nabla f(x) = Ax - b, \tag{16.3}$$

also known as the *residual* (for reasons to be introduced in §17.2). The terms residual and gradient will be used interchangeably, depending on which aspect is to be emphasised. $f(x)$ has constant Hessian matrix $\nabla\nabla^\intercal f(x) = A$. Because A is SPD, it has a unique inverse, which will play such a central role that it will be afforded its own symbol:

$$A^{-1} =: H.$$

(The choice of the letter H is historic convention for inverse Hessians in optimisation.) Problem (16.1) is interesting for two reasons. First, it lies at the heart of least-squares estimation, which in turn is the basis for many basic numerical, statistical, and indeed machine learning algorithms, like Gaussian process regression (§4.2). Second, as we will see in Chapter IV, estimating SPD matrices and their inverse also features in local nonlinear optimisation. In fact, some of the most core algorithms of nonlinear optimisation will arise as natural extensions of the results of this chapter.

Solving linear problems is also interesting more generally because certain other core tasks of linear algebra can be solved with algorithms that are structurally similar. In particular, there is a deep connection between iterative linear solvers and methods for computing matrix decompositions (like eigen- and singular value decompositions) through the Krylov sequence.[2] Developing a probabilistic description for (iterative) linear solvers thus holds the promise of similar advances in other linear algebra tasks.

The largest part of this chapter will be devoted to re-phrasing an existing, elementary linear algebra method – the method of conjugate gradients. The point is to build an intuition for the constraints on the model class, in particular on the choice of

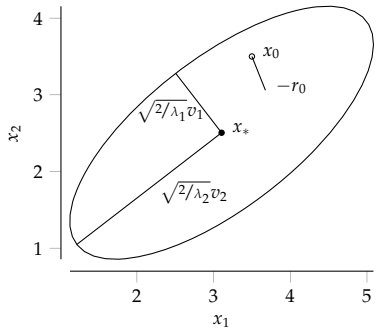

Figure 16.1: Sketch of a symmetric positive definite linear problem.

[2] See §11.3 in Golub and Van Loan (1996) for an introduction. This connection is also the reason why computing the posterior uncertainty (covariance) of Gaussian process regression adds little to no overhead to just computing the associated point estimate (posterior mean). Finding the minimum of a quadratic problem requires the same considerations as characterising the geometry (Hessian) of the problem. As we have already seen in previous chapters, this has crucial implications for the complexity of Probabilistic Numerics: if we are happy to accept approximately Gaussian posteriors, then these can often be had at essentially no extra cost over the classic point estimate.

https://doi.org/10.1017/9781316681411.022 Published online by Cambridge University Press

prior covariance, that are required to build a lightweight, efficient estimator for linear algebra quantities. Because they form the foundation of many computational methods, linear solvers have to work on a large class of problems at low computational cost. For the most basic methods, this also means they must not themselves require the solution of a linear problem, not even one smaller than the original one. This restriction is less important for practical applications requiring uncertainty, where one may be willing to pay a small computational overhead for probabilistic "error bars" on point estimates. In such situations, it may be convenient to use a classic (highly optimised) solver for internal computations, and construct a probabilistic estimate "around" its output, using some statistics of the solver's computations to calibrate uncertainty.

▶ 16.1 Classic Methods

The "pedestrian" solution to Problem (16.1) is *Gaussian elimination*,[3] one of the oldest algorithms known to humankind, and certainly one the best known ones, widely taught at high-school level. Gaussian elimination provides a solution for *any* simultaneously solvable set of linear equations, not just symmetric positive definite ones. It is the standard approach for small and medium-sized problems, among other reasons because it can be made numerically quite stable by ordering the operations suitably.[4] Gaussian elimination of an $N \times N$ system in general involves N intermediate steps, each involving a matrix-vector multiplication, hence the overall complexity is $\mathcal{O}(N^3)$.

[3] See Gauss (1809). For a historical perspective, see Grcar (2011).

However, Gaussian elimination is not an "anytime" algorithm – it only provides a correct answer after it runs to completion. In fact it can be shown[5] that if this algorithm is stopped at step $i < N$ and the computations performed until this point are used to compute an "estimate" $\hat{x}_i$ of the true solution x, then there are scenarios in which the error $\|\hat{x}_i - x\|$ can actually *grow* with every step $i < N$, and only converge suddenly on the final step $i = N$. When studying probabilistic formulations of linear algebra, we will primarily be interested in assigning uncertainty in incompletely solved problems, and thus in algorithms which improve a point estimate over the course of their run.

[4] In formal terms, Gaussian elimination is often captured in the so-called LU-decomposition, a notion introduced by Turing (1948). The decomposition is made numerically stable by helpful permutations of A, a process known as *pivoting*. Details can be found in §3.2–3.4 of Golub and Van Loan (1996). The LU decomposition splits the matrix A into a **L**ower and an **U**pper triangular matrix, and is a generalisation of the Cholesky decomposition (Benoit, 1924) (which only applies to SPD matrices). More can be found in Turing's very readable work.

[5] Hestenes and Stiefel (1952)

https://doi.org/10.1017/9781316681411.022 Published online by Cambridge University Press

```
1  procedure CG(A(·), b, x_0)
2      r_0 = Ax_0 − b                          // initial gradient
3      d_0 = 0, β_0 = 0                        // for notational clarity
4      for i = 1, ..., N do
5          d_i = −r_{i−1} + β_{i−1} d_{i−1}    // compute direction
6          z_i = A d_i                         // compute
7          α_i = −d_i^T r_{i−1} / d_i^T z_i    // optimal step-size
8          s_i = α_i d_i                       // re-scale step
9          y_i = α_i z_i                       // re-scale observation
10         x_i = x_{i−1} + s_i                 // update estimate for x
11         r_i = r_{i−1} + y_i                 // new gradient at x_i
12         β_i = r_i^T r_i / r_{i−1}^T r_{i−1}  // compute conjugate correction
13     end for
14 end procedure
```

Algorithm 16.1: Conjugate gradients (basic form). Adapted from Nocedal and Wright (1999). The algorithm takes as inputs the problem description (A, b) and an initial guess x_0 (often set to either $\mathbf{0}$ or b), then iteratively computes estimates x_i of typically increasing quality. The dominant computation is the matrix-vector multiplication in line 6, all other steps are of either linear or constant cost. Lines 8 and 9 could obviously be folded into the following two lines, they are kept separate here to ease the comparison to probabilistic equivalents constructed below.

Iterative solvers provide such a behaviour, and are used for large-scale problems where the cubic cost of finding the exact solution is too large to contemplate. These methods also involve individual steps of cost $\mathcal{O}(N^2)$ each. But they aim to continuously improve an initial guess $\hat{x}_0$, so that the algorithm can be stopped for $i \ll N$ and already provide a good estimate of x. The prototypical algorithm in this class is the *method of conjugate gradients* (CG),[6] an algorithm applicable to the SPD problem (16.1) above. For reference, it is displayed in Algorithm 16.1.

[6] Hestenes and Stiefel (1952)

CG consists of a single loop of iterative refinements to an estimated solution x_i of Eq. (16.1). Each iteration computes a single matrix-vector multiplication (line 6), a projection of A along a vector d_i. The other lines consist of cheaper computations that track the current estimate x_i and the current residual $r(x_i)$. The steps are determined by two "control" parameters α, β. The former, α, regulates how the observed result of the projection is used to update x_i, and latter, β, governs which projection is chosen next. Hence, this algorithmic structure fits with the separation of an active agent into an estimation (inference) and action (decision, policy) part that we already employed in the integration chapter.

The central result of this chapter on linear algebra will be the construction of a set of active probabilistic inference methods that are consistent with conjugate gradient. That is, they construct a sequence of Gaussian posterior measures associated with a mean estimate for x that equals the sequence constructed by CG. Even for readers interested in non-symmetric,

https://doi.org/10.1017/9781316681411.022 Published online by Cambridge University Press

non-definite problems, CG is a good starting point, as it can be generalised to larger classes of tasks. Many iterative solvers can be constructed in this way, including the *generalised minimal residual* method (GMRES)[7] and the *bi-conjugate gradient* method (BiCG),[8] the *conjugate gradient squared* method (CGS),[9], and the *quasi-minimal residual* method (QMR).[10] Further, CG is closely connected to the *Lanczos processLanczos process* for approximately computing the eigenvalues of symmetric matrices,[11] and the generalisation to GMRES is analogously connected to the more general *Arnoldi process* for iterative approximation of eigenvalues.[12] A probabilistic analysis of CG thus gets us closer to a general understanding of linear algebra in terms of probabilistic inference.

[7] Saad and Schultz (1986)
[8] Fletcher (1975)
[9] Sonneveld (1989)
[10] Freund and Nachtigal (1991)
[11] Lanczos (1950)
[12] Arnoldi (1951)

There are several good textbooks available for readers interested in the classic analysis of linear algebra methods. German-speaking readers can find a great overview of all the methods mentioned above, and of their relationship to each other, in §4.3 of the book by Meister (2011). A more compact introduction to GMRES and CG in particular is also in §IV of Trefethen and Bau III (1997). The book by Parlett (1980) specifically focuses on symmetric matrices; the contents of the present chapter are related to the exposition in Chapters 12 and 13 therein.

https://doi.org/10.1017/9781316681411.022 Published online by Cambridge University Press